Simulation-informed transfer learning improves low-data nanobody thermal stability prediction

Taihei Murakami[†1], Kentaro Sasaki[†1], and Yasuhiro Matsunaga[*1,2]

[1]Saitama University, Saitama, Japan
[2]RIKEN Center for Computational Science, Kobe, Japan

[†]These authors contributed equally to this work.

Nanobody development often depends on thermal stability, but experimental melting-temperature (Tm) measurements remain scarce. Simulations and structure-based calculations can label many mutated or designed sequences, yet these labels usually measure related quantities rather than Tm itself. We tested whether such computational labels can improve low-data nanobody Tm prediction under strictly experimental model selection. Using a public nanobody Tm benchmark, we trained transfer-learning models that combined Tm prediction with one computational prediction task at a time; checkpoints, model choices, and final evaluation used only experimental Tm validation or test examples. We compared mutation free energies from alchemical free-energy perturbation (FEP), Rosetta mutation scores, ThermoMPNN stability scores, Rosetta scores on ESM2-generated or random variants, and an MD-derived Q-value summarizing native-contact persistence. FEP gave the clearest benefit: held-out Tm mean absolute error decreased from 6.61 to 6.26°C relative to Tm-only training, with a paired improvement of 0.35°C (90% confidence interval, 0.24–0.46°C). The reduction was small in absolute magnitude but robust in the paired test-set analysis. The gain was not reproduced by larger protein sequence models or by the MD-derived Q-value, whereas ESM2 variants scored by Rosetta gave a weaker positive signal. Thus, computational labels can improve nanobody Tm prediction when their physical meaning transfers to the experimental endpoint; mutation free energies complement sparse absolute Tm measurements.

nanobody thermostability | melting temperature | simulation-to-real transfer learning | protein language model | free energy perturbation

Correspondence: Yasuhiro Matsunaga, ymatsunaga@riken.jp

Introduction

Simulation data for low-data experimental prediction

Machine learning has made it possible to screen large molecular and materials spaces before costly experimental testing, but its usefulness still depends on the availability of reliable property labels. In materials informatics, this data bottleneck has motivated a simulation-to-real (Sim2Real) strategy: large computational databases generated by first-principles calculations, quantum chemistry, or molecular dynamics simulations are used as source data, and the resulting models are transferred to smaller experimental target datasets (1–3). A recent study in polymers and inorganic materials showed that the downstream prediction error of transferred models can scale with the amount of computational data, providing a quantitative way to judge whether a growing simulation database has real experimental value (3). This framing is important because simulation data are not useful by default: simulations and experiments are separated by model assumptions, sampling limits, and systematic domain gaps.

Protein engineering faces an analogous problem, but with an additional layer of physical mismatch. Experimental measurements of protein properties are expensive and sparse, whereas biomolecular simulations and physics-based calculations can generate labels for many designed or mutated sequences. The challenge is sharper than in many property-matched transfer settings, because the simulated quantity may not be the same as the experimental target. For example, a target phenotype may be the melting temperature (Tm), the temperature at which a protein loses half of its folded population under a given assay condition. A tractable calculation, by contrast, may estimate how much a mutation changes stability, assign a structure-based stability score, or summarize a molecular-dynamics trajectory. The central question is therefore not simply whether simulation data can be added to a training set, but which simulated physical quantities can transfer to the experimental phenotype.

Nanobody thermostability in the high-throughput era

We examine this question using nanobody thermostability as a concrete case. Nanobodies, or single-domain VHH antibodies, are attractive engineering scaffolds because of their small size, modularity, and binding properties (4, 5). Their practical use, however, depends on thermal stability during expression, storage, formulation, and downstream screening. Experimental Tm measurements remain limited, and direct molecular simulation of thermal stability is computationally demanding (6). Sequence-based predictors and protein language models (PLMs) offer an attractive alternative: PLMs such as ESM2 learn broadly useful protein representations from large sequence corpora (7, 8), and recent methods have used deep sequence representations to predict protein or nanobody melting temperatures (9–11). High-throughput stability measurements are also changing the data landscape. Tsuboyama et al. introduced a cDNA-display proteolysis assay and used it to pro-

duce a mega-scale folding-stability dataset with approximately 776,000 high-quality measurements from natural and designed protein domains (12). That advance shows that experimental ΔG can now be measured at unprecedented scale for suitable small domains. However, the curated measurements span 40–72 amino-acid domains, whereas nanobody VHH domains are typically much longer, and comparable high-throughput ΔG measurements are not yet available for full-length nanobody VHH domains. A model fine-tuned on the mega-scale dataset also showed limited transfer to larger protein scaffolds: although it performed well on small protein domains, benchmarks on 177–501-residue proteins were substantially weaker than structure-based methods (13). Thus, large-scale ΔG data are powerful but do not remove the need for target-specific supervision and careful validation in nanobody Tm prediction (14–16).

Study design and main finding

Here we ask whether simulation-derived labels can provide that additional supervision when experimental Tm labels are scarce. We treat Tm prediction as the experimental target task and treat each computational label as a source task that can shape the learned sequence representation. The implementation uses ESM2, a pretrained protein language model, as the shared sequence encoder (8). The computational labels represent several kinds of computational information: mutation free-energy changes from alchemical free-energy perturbation (FEP) calculations, mutation scores from Rosetta, stability scores from ThermoMPNN, Rosetta scores on ESM2-generated and random variants, and a molecular-dynamics (MD)-derived Q-value measuring native-contact persistence. To keep the comparison target-centered, all model settings are selected using an experimental validation set, and all final claims are evaluated on the same held-out experimental Tm test set with paired error comparisons. This design prevents high performance on a computational task from being mistaken for improved Tm prediction.

The main result is that the benefit of simulation-derived supervision is highly source-dependent. Mutation free-energy labels from FEP provide the strongest improvement over training on Tm labels alone, whereas the MD-derived Q-value does not improve held-out Tm prediction under the same evaluation design. ESM2 variants scored by Rosetta provide a weaker but positive signal, suggesting a connection to future generator–physics–predictor design loops. Taken together, these results extend the Sim2Real argument from materials informatics to a biomolecular setting in which the source and target labels differ in physical meaning. Useful simulation data are not merely abundant; they encode a physical quantity that transfers to the experimental property of interest.

Materials and Methods

Experimental and technical design

This study was designed to test whether computational source labels can improve experimental nanobody melting-temperature (Tm) prediction when the source label is physically related to, but not identical with, Tm. We therefore used a multi-task transfer-learning design: experimental Tm prediction was the target task, and computational labels were source tasks that shared part of the neural network with the target task. The central constraint was that all model selection and all final claims were anchored to experimental Tm performance. Source-task losses were used for representation learning during training, but source-task validation errors were not used to select final checkpoints or final hyperparameters.

Experimental Tm data set and split definitions

Experimental Tm labels were taken from a public nanobody benchmark data set, NbBench, which provides a thermo-seq task with predefined splits (15). The processed files used in this study contained 57 training sequences, 114 validation sequences, and 396 test sequences. The terms were used as follows. The training split was used to fit model parameters. The validation split was used for checkpoint selection, early stopping, and hyperparameter selection. The test split was held out until the selected settings were fixed and was used only for final reporting. This definition applies to both Tm-only models and multi-task transfer models.

The target CSV files contained an amino-acid sequence column and a Tm label in degrees Celsius. During training, Tm labels were transformed to a scaled regression target. The scaler was fit only on the 57 training labels, using a robust scaling step followed by min–max scaling to [0,1]. The fitted scaler was then applied to validation labels during training. At evaluation time, model predictions were inverse-transformed back to degrees Celsius before computing errors. The validation and test label distributions were therefore not used to fit the target-label scaling transform.

For the experimental-label scaling reference, the Tm training split was subsampled to the requested number of labels using the run seed. The validation and test splits were not subsampled. These experiments provide a target-label reference for interpreting whether computational source-label scaling behaves in the expected direction.

Computational source-label data sets

Computational labels were represented as source tasks rather than as additional Tm measurements. The source-label collection comprised mutation free-energy changes from alchemical free-energy perturbation (FEP), structure-based mutation-effect scores from Rosetta, learned mutation-effect scores from ThermoMPNN, Rosetta scores assigned to generated and

random variants, and a molecular-dynamics (MD)-derived Q-value label that summarizes native-contact persistence. All source labels were preprocessed to [0,1]-scaled regression targets and were used only through their own source-task heads.

Mutation-effect sources were organized as two template-specific tables derived from the 1MEL and 4IDL nanobody structures. FEP, Rosetta, and ThermoMPNN each contained 435 and 409 processed rows for the two templates. ESM2-generated variants and random variants scored by Rosetta each contained 1000 and 1000 processed rows. The MD-derived Q-value source was generated from a separate panel of simulated single-domain antibody structures selected from SAbDab, the Structural Antibody Database (17). The processed MD table contained 1143 analysis-ready rows. Row counts exclude headers.

We checked exact sequence overlap between source-label tables and the NbBench splits. The mutation-effect and generated-variant source tables with explicit sequence columns had no exact overlap with the NbBench training, validation, or test sequences. The SAbDab-derived MD table contained a small number of exact matches to NbBench sequences, including eight test-set sequences; because the MD Q-value source did not improve final Tm prediction, this overlap does not support the positive FEP result but is reported for transparency.

For a mutation-effect experiment with requested source-label count n, up to n rows were sampled independently from each of the two template-specific source tables using the run seed. The two template-specific tables were assigned separate task identifiers. For an MD-derived Q-value experiment with requested source-label count n, n rows were sampled from the single processed Q-value table. After sampling, source-label rows were split 80/20 into source-training and source-validation rows using the same run seed. These source-validation rows were retained for monitoring and control analyses. In the reported final protocol, however, the validation data passed to the trainer for checkpoint selection and early stopping were filtered to the experimental Tm validation rows.

SAbDab-derived MD simulations and Q-value labels

The MD source-label set was built from nanobody structures selected from a SAbDab-derived summary table (17). We selected one heavy-chain nanobody structure per PDB entry when an IMGT-renumbered coordinate file was available and the selected chain could be verified in that file. The 400 K simulation panel contained 1146 nanobody systems. After requiring a usable trajectory, topology, reference structure, sequence of at least 50 residues, and at least one selected native contact, 1143 trajectories entered the processed source-label table.

MD systems were prepared with MDClaw using the selected nanobody chain only. The IMGT PDB file was registered as the structure source, the heavy chain was extracted, non-protein components were excluded, termini were left uncapped, and protein protonation was assigned at pH 7.4. The preparation workflow used PDBFixer for structure cleaning and missing-atom handling, followed by pdb2pqr/PROPKA for pH-aware protonation and Amber-compatible atom naming when available (18, 19). Systems were solvated in explicit OPC water with 15 Å padding and 0.15 M NaCl, and Amber topologies were built with the ff19SB protein force field and OPC water model (20–22).

The trajectory protocol used OpenMM on CUDA devices (23). Each system was first equilibrated at 300 K with 1 ns NVT followed by 1 ns NPT at 1 atm, using a 2-fs timestep without hydrogen-mass repartitioning. The 400 K branch was then generated from the 300 K equilibrated state by two restrained NVT relaxation stages: 1 ns at 400 K with Cα restraints followed by a second 1 ns 400 K NVT restrained relaxation. The production trajectory used 100 ns of restraint-free 400 K NVT dynamics, hydrogen-mass repartitioning with 4 amu hydrogen mass, a 4-fs timestep, Langevin middle integration with 1 ps^{-1} collision rate, PME electrostatics with a 1.0-nm nonbonded cutoff, and coordinate/energy reporting every 100 ps. The reported MD source labels use the 400 K production branch.

Q-values were extracted from the 400 K production trajectories with MDTraj (24). For each trajectory, the simulation topology was loaded from the Amber parameter file and the native reference was defined as frame 0 of the production trajectory. A smooth native-contact fraction (25) was computed from backbone-heavy-atom contacts and averaged over the terminal portion of the trajectory. The raw trajectory averages were min–max scaled to [0,1] before training. The exact contact selection, contact function, averaging window, and processed-column convention are reported in Supplementary Methods.

FEP mutation free-energy labels

The FEP source labels were mutation free-energy changes for variants associated with the 1MEL and 4IDL nanobody template sequences. FEP calculations followed the same protocol as our related nanobody thermostability study. Briefly, alchemical mutation simulations were run with NAMD 3 using GPU acceleration (26). Input systems were prepared in VMD (27) using a customized AlaScan workflow (28). Simulation coordinates were initialized from AlphaFold 3 models of the corresponding wild-type sequences (29). Systems used the CHARMM36 protein force field (30), TIP3P water (31), and 150 mM NaCl.

Each alchemical transformation used a dual-topology mutation setup with a hybrid-residue topology for CHARMM36. The transformation was divided into 20 equally spaced λ windows from wild type to mutant. Forward and backward transformations were simulated, with 1 ns per direction, a 2-fs integration timestep, and

200 ps equilibration per window before data collection. Simulations were performed in the NPT ensemble at 300 K and 1 atm with Langevin temperature control and a Langevin piston barostat. Bonds involving hydrogen atoms were constrained with SHAKE (32), and long-range electrostatics were evaluated with particle mesh Ewald (33). Relative free energies were estimated with the Bennett acceptance ratio (34). The processed learning labels used the sign convention and [0,1] scaling stored in the FEP CSV files.

Rosetta and ThermoMPNN mutation-effect labels

Rosetta mutation-effect labels were generated with the Rosetta ddg_monomer application (35). For each template, the PDB structure was cleaned to the target chain, variant sequences were compared with the corresponding wild-type sequence, and the sequence differences were converted to Rosetta mutation-list files. The calculation used full-atom Rosetta with the ref2015 score function (36), 50 iterations, Cartesian minimization enabled, local-only optimization disabled, sidechain-minimization-only disabled, minimum-score aggregation, and no PDB dumping. The same Rosetta protocol was used for the template mutation-effect tables, the ESM2-generated variants, and the random-variant control.

For the generated-variant source, ESM2 650M was used to sample 100,000 two-mutation variants from each wild-type template sequence, variants were ranked by pseudo-log likelihood, and the top 1% were retained for Rosetta scoring (8). The random-variant control used approximately 1000 two-mutation variants generated by randomly choosing mutation positions and amino acid substitutions, followed by the same Rosetta scoring workflow.

ThermoMPNN source labels were computed from the same template-specific variant sets using ThermoMPNN, a stability-change predictor trained by transfer learning from larger protein-stability data (37). The ThermoMPNN outputs were converted to the same processed CSV format as the other mutation-effect source tasks.

Neural-network architecture

All models used an ESM2 protein language-model encoder (8) followed by lightweight regression heads. Unless otherwise stated, the encoder was facebook/esm2_t6_8M_UR50D. Sequences were tokenized with truncation and padding to a maximum length of 160 residues. The first-token encoder representation was passed to a shared multilayer perceptron with hidden dimensions 256, 128, and 32, ReLU activations, and dropout. A scalar Tm head predicted the scaled target Tm label. In source-transfer models, additional scalar source heads predicted the source-task labels from the same shared representation.

The Tm-only baseline used the ESM2 encoder, the shared trunk, and the Tm head with only experimental Tm labels. Source-transfer models used the same target path plus source-task heads. The main FEP model used separate source heads for the two template-specific mutation-effect tasks, which was the best-performing source-head design in the source-head comparison. Additional controls evaluated a shared mutation-effect head, a shared head conditioned on a template embedding, and a shared head with template-specific affine calibration. Architecture controls also evaluated residual and latent source-fusion variants, but the main result used the original shared-trunk architecture.

Fine-tuned-encoder and frozen-encoder training

Two encoder-training regimes were evaluated. In the fine-tuned-encoder regime, the ESM2 encoder and regression heads were optimized jointly. The encoder used a separate learning rate from the newly initialized trunk and heads, so that the pretrained representation was fine-tuned more conservatively than the output layers. In the final FEP setting, the selected encoder learning rate was 3×10^{-5} and the head learning rate was the default 3×10^{-4}.

In the frozen-encoder control, all ESM2 parameters were fixed by setting their gradients off. The encoder was run once to precompute sequence embeddings for the target and source examples, and training then optimized only the downstream trunk and task heads. This control asks whether the source labels help when the pretrained sequence representation is used purely as a fixed feature extractor. Encoder-size controls repeated selected Tm-only and FEP-assisted settings with 35M- and 650M-parameter ESM2 encoders.

Loss function and multi-task objective

Each training example was represented as (x_i, y_i, t_i), where x_i is the sequence, y_i is the scaled regression label, and t_i is the task identifier. The target Tm task used one task identifier, and each source task used its own identifier. A minibatch could contain target and source examples, but each example contributed loss only through the head for its task.

The per-task regression loss was Huber loss with $\delta = 1.0$ on scaled labels. The default multi-task objective used learnable task-uncertainty weights (38). For task k with loss L_k and learned log-scale s_k, the contribution was

$$\frac{L_k}{2\exp(2s_k)} + s_k.$$

The total loss was the sum over the tasks present in the minibatch. Fixed source-task weights were also evaluated as controls, particularly for the MD-derived source labels.

Training, early stopping, and hyperparameter selection

Models were trained with AdamW, cosine learning-rate scheduling, batch size 16, weight decay 0.04, 100 warmup

steps, and at most 400 epochs. The default head learning rate was 3×10^{-4}, the default fine-tuned-encoder learning rate was 1×10^{-4}, and the default dropout rate was 0.15. Training used mixed precision on CUDA devices. Checkpoints were saved and evaluated once per epoch.

Within each run, the best checkpoint was the checkpoint with the lowest mean squared error on the experimental Tm validation examples. Early stopping used the same validation metric with a patience of 15 epochs. In source-transfer models, source-label rows were present in the training set, but they were removed from the validation data passed to the trainer for checkpoint selection and early stopping. This is the implementation-level step that ensures that the reported checkpoints are selected by target-task validation performance rather than by source-task performance.

Hyperparameter selection was also performed on the experimental Tm validation split. Candidate settings varied encoder learning rate, head learning rate, dropout, loss-weighting mode, source-head design, and selected architecture controls. The search included encoder learning rates of 3×10^{-5}, 1×10^{-4}, and 3×10^{-4}; a lower head learning rate of 1×10^{-4} paired with encoder learning rate 3×10^{-5}; dropout rates of 0.05, 0.15, and 0.30; and fixed source-task loss weights where relevant. Each candidate setting was evaluated as a three-seed ensemble on the Tm validation split. The selected setting for each source class was then evaluated on the held-out test split as a ten-seed ensemble.

Independent run seeds controlled model initialization, target-training subsampling in the target-label scaling experiment, source-label sampling, and source-task train/validation splitting. Final comparisons used seeds 1–10. The label-count curves used fixed source-specific training recipes while varying the number of source labels, rather than retuning a separate model for every label-count point.

Final evaluation and statistics

Final evaluation used the same 396 held-out Tm test examples for all conditions. For each condition, the ten independently seeded models produced scaled Tm predictions for every test sequence. Predictions were averaged in scaled-label space and then inverse-transformed to degrees Celsius. The primary metric was mean absolute error (MAE) in degrees Celsius. Root mean squared error, coefficient of determination, Spearman correlation, and Pearson correlation were also computed, but MAE was used for the main comparisons because it is directly interpretable as average temperature error.

Confidence intervals were estimated by nonparametric bootstrap resampling over held-out test proteins. For a single condition, the absolute-error vector from the ten-seed ensemble was resampled with replacement 10,000 times using a fixed random seed. The reported 90% confidence interval is the 5th to 95th percentile range of the bootstrapped MAE values.

For paired comparisons, the same bootstrap indices were applied to the absolute-error vectors from the candidate and reference conditions. The candidate-minus-reference MAE difference was then computed for each bootstrap sample. This paired bootstrap preserves the matched-test-example structure of the comparison. In figures and result tables, ΔMAE is candidate minus reference, so negative values indicate lower error than the reference. The one-sided tail probability stored in the result JSON is the fraction of paired bootstrap samples in which candidate-minus-reference ΔMAE is greater than zero. The manuscript emphasizes effect sizes and 90% confidence intervals.

Implementation details

The training and evaluation pipeline is implemented in prepare.py and train.py. The data loader constructs target and source tasks, applies target-label scaling, performs seeded source-label sampling, filters validation data for target-task checkpoint selection, evaluates seed ensembles, and writes bootstrap statistics. Each result directory contains a machine-readable scaling.json file with command-line arguments, resolved hyperparameters, per-point metrics, bootstrap intervals, and paired comparisons.

Results

A target-centered framework for using simulation labels in Tm prediction

We first established an evaluation framework for asking whether simulation-derived labels improve experimental Tm prediction. The target task was nanobody Tm regression from amino-acid sequence. Computational labels were treated as auxiliary prediction tasks rather than as substitutes for experimental Tm. Each model used a pretrained protein sequence model, ESM2, to convert a sequence into a learned representation. The representation was shared by a Tm prediction head and a computed-label head, allowing the computational task to influence the representation while keeping the final objective anchored to experimental Tm prediction (Fig. 1).

The comparison was deliberately target-centered. All candidate settings were selected using an experimental Tm validation set, and the final numbers were computed only on a held-out experimental Tm test set. Improvements were assessed by paired differences in absolute error on the same test examples. This design addresses a common ambiguity in simulation-to-experiment learning: a computational task can be learnable and physically meaningful without improving the target experimental phenotype.

Scaling depends on the physical label

We next varied the number of experimental and computational labels to determine whether more source data

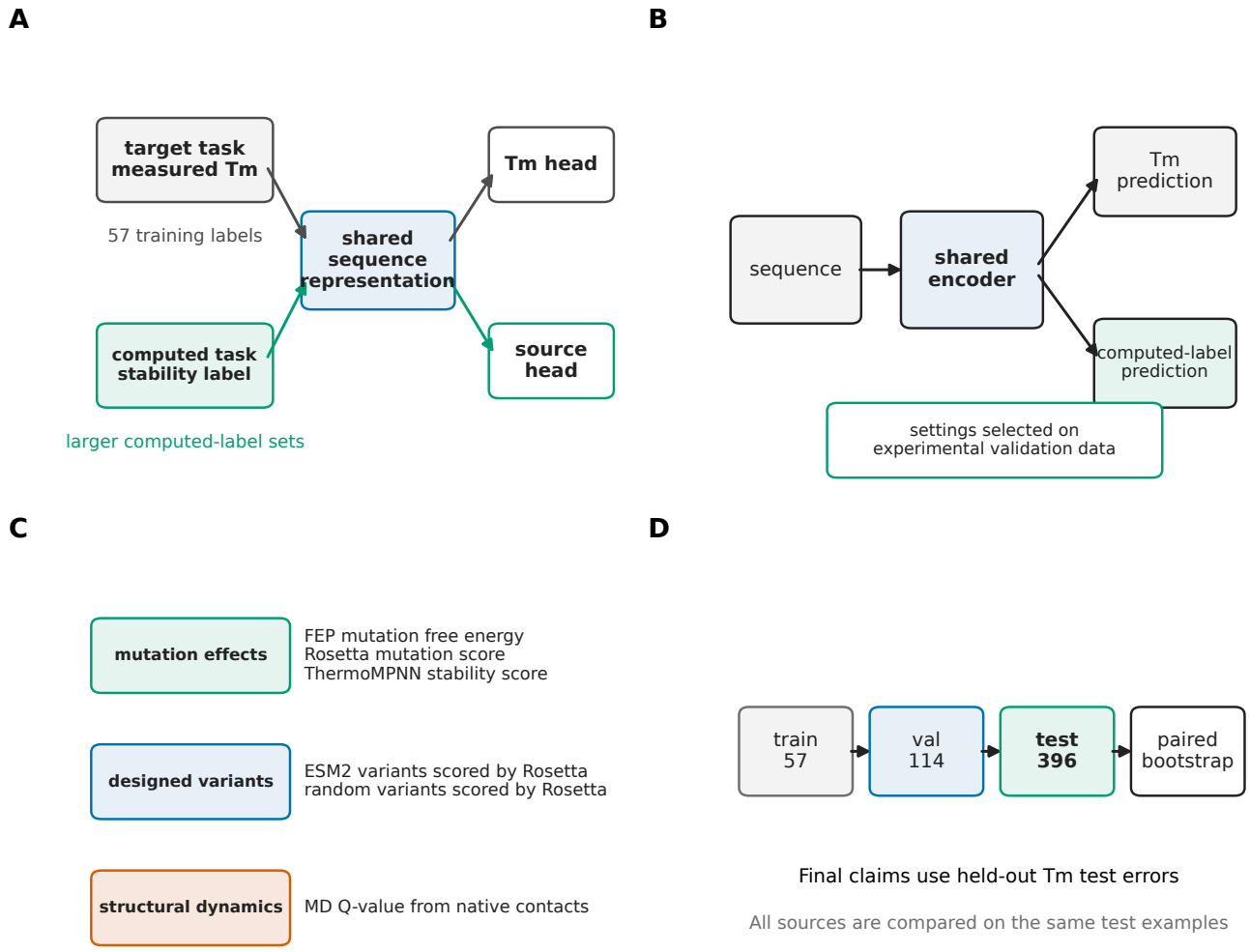

Fig. 1. Multi-task transfer-learning framework for using simulation data in experimental Tm prediction. (a) Experimental Tm labels are scarce, whereas simulation and physics-based calculations can provide larger computed-label pools. (b) Nanobody Tm prediction is used as a concrete case in which the computed labels are related to, but not identical with, the experimental target. (c) The model shares a pretrained protein sequence representation between the Tm target head and a computed-label head. (d) Model settings are selected on an experimental validation set, and final claims are evaluated on held-out experimental Tm test examples using paired error statistics.

translated into better Tm prediction. As expected, increasing the number of experimental Tm labels improved the model trained only on Tm measurements (Fig. 2). The target-label sweep used 10, 20, 30, 40, and 57 experimental Tm training labels. The FEP sweep used 10, 40, 80, 160, and 320 rows per template-specific source table, and the MD Q-value sweep used 10, 40, 80, 160, 320, and 640 rows from the single MD source table. Mutation free-energy labels from FEP calculations showed a beneficial trend: the largest FEP label-count setting tested gave the best held-out Tm error among the FEP settings. The curve was not strictly monotonic, so we do not interpret it as a clean power law. Instead, we interpret it as evidence that additional mutation free-energy labels can improve the target task when they are selected and evaluated through the experimental Tm endpoint.

The MD-derived Q-value label behaved differently. This label measured how strongly native contacts persisted during high-temperature MD trajectories. Increasing the number of these labels did not yield a robust improvement over learning from Tm measurements alone, even though the computed labels were also generated from simulations. Thus, the label-count experiment already suggested the central result of the paper: the number of computed labels is not sufficient. The transferable physical content of the label matters.

Mutation free-energy labels provide the strongest final improvement

We then compared the selected settings for each computational label. Training on the experimental Tm labels alone achieved a held-out test MAE of 6.61°C. Adding FEP mutation free-energy labels reduced the MAE to 6.26°C, corresponding to a source-minus-Tm-only paired change of -0.35°C (90% confidence interval,

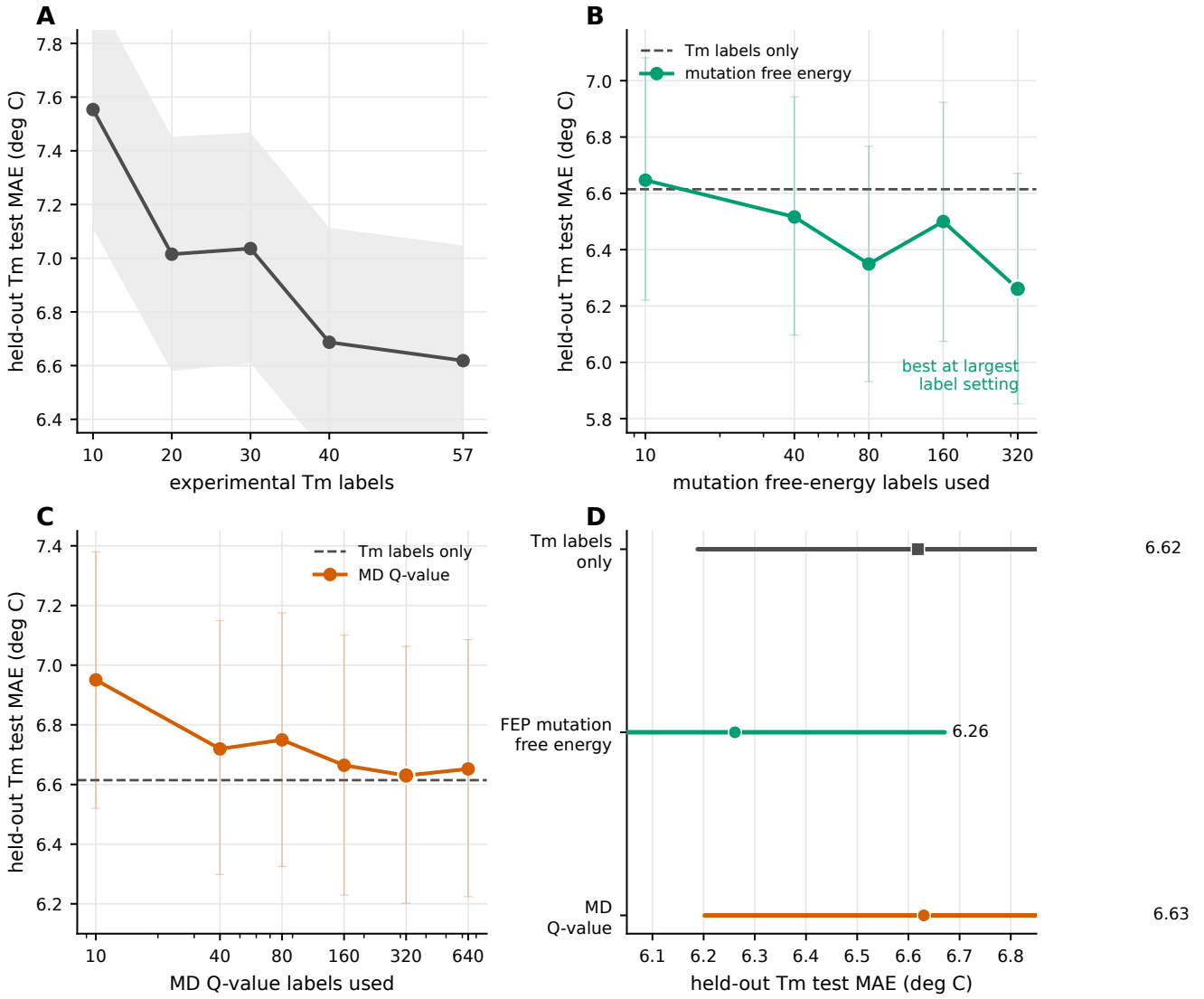

Fig. 2. Label-count scaling of experimental and computational supervision. (a) Learning from experimental Tm labels alone improves as more Tm labels are used. The shaded band shows the 90% bootstrap interval. (b) Mutation free-energy labels from FEP reach their best held-out Tm error at the largest computed-label setting tested, although the curve is not strictly monotonic. Points show ensemble MAE values and thin vertical intervals show 90% bootstrap intervals. (c) The MD-derived Q-value does not show a robust improvement over learning from Tm labels alone. The confidence intervals are shown with the same thin-interval convention as in panel b. (d) Best points from the label-count sweeps are replotted as interval estimates to compare the experimental-only, FEP, and MD Q-value references.

-0.46 to -0.24°C; Fig. 3). Negative paired changes indicate lower error than the Tm-only reference. This was the clearest improvement observed across all computational labels. The absolute gain is incremental relative to the 6.61°C baseline error, but it is consistent across paired test examples and across the bootstrap interval.

The other sources showed weaker or negative effects. Rosetta scores assigned to ESM2-generated variants reached 6.45°C (paired change, -0.17°C; 90% confidence interval, -0.31 to -0.03°C), and ThermoMPNN stability scores reached 6.46°C (paired change, -0.15°C; 90% confidence interval, -0.30 to -0.01°C). Rosetta mutation scores reached 6.53°C (paired change, -0.09°C; 90% confidence interval, -0.20 to +0.02°C), and random variants scored by Rosetta reached 6.51°C (paired change, -0.10°C; 90% confidence interval, -0.22 to +0.03°C). Their intervals crossed zero. The MD-derived Q-value worsened the final test error to 6.73°C (paired change, +0.12°C; 90% confidence interval, +0.01 to +0.22°C). These results support a selective, source-dependent interpretation: mutation free-energy-like labels were useful, but an MD Q-value summarizing native contacts was not sufficient for this target task.

The FEP gain is not explained by sequence-model size alone

Because the size of a protein language model can affect downstream performance, we tested whether the benefit of FEP labels could be reproduced simply by using a larger sequence encoder. It could not. In the experimental-only setting, larger 35M and 650M encoders did not improve held-out performance relative

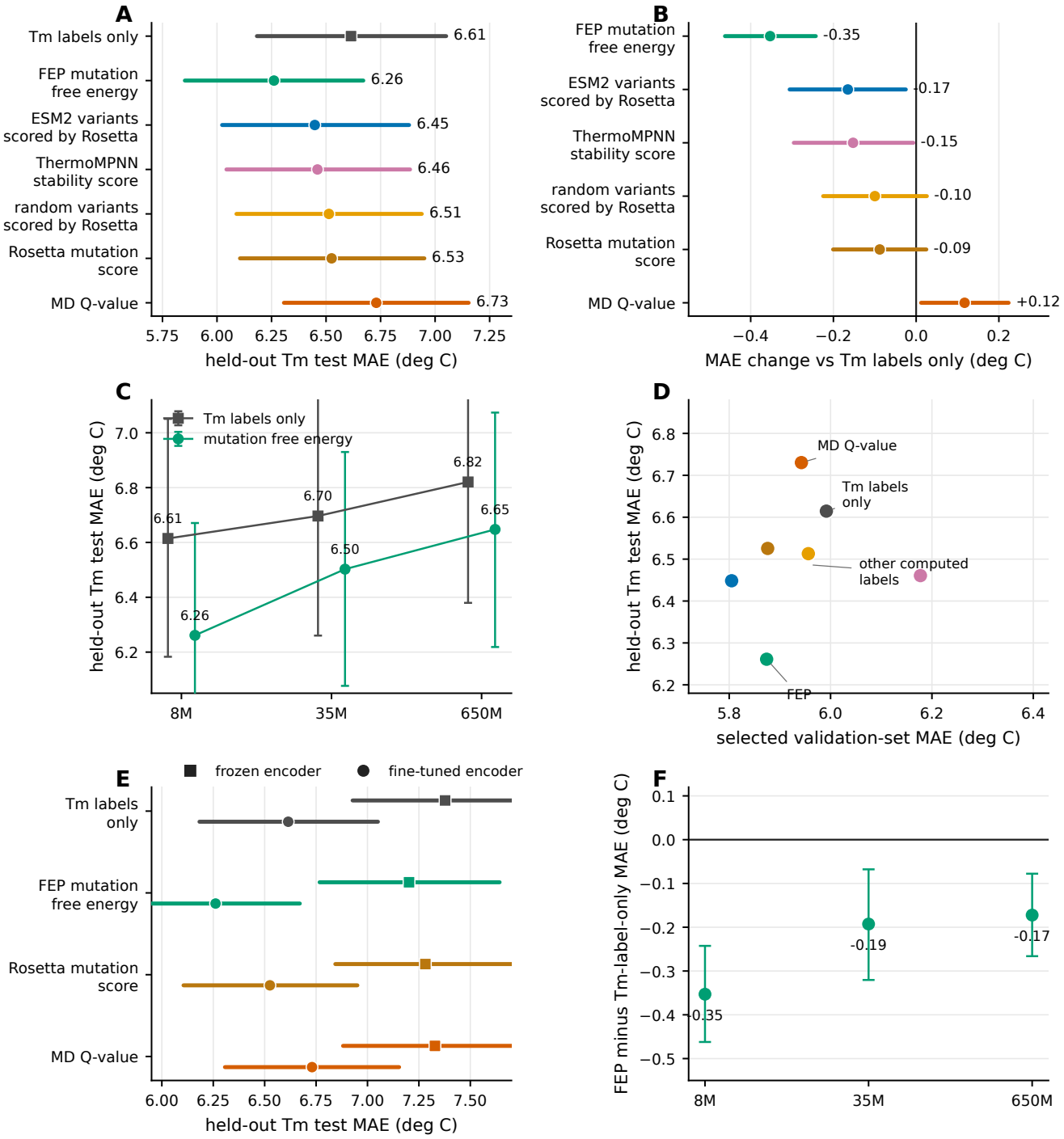

Fig. 3. Final held-out Tm performance across computational labels and controls. (a) Final test MAE for training on experimental Tm labels alone and for each computational label. Intervals show bootstrap confidence intervals over test-set absolute errors. (b) Paired ΔMAE relative to training on Tm labels alone. Negative values indicate improvement. (c) Protein sequence model size controls show that larger encoders do not improve the selected experimental-only or FEP settings. (d) Validation-set performance and held-out test performance are related but not identical, emphasizing the need for final target-task testing. (e) Frozen-encoder and fine-tuned-encoder comparisons show that FEP helps in both regimes, with the strongest absolute performance obtained when the sequence representation is fine-tuned. (f) Paired FEP gains remain negative across protein sequence-model sizes, but the best absolute error is obtained with the 8M encoder.

to the 8M encoder. In the FEP setting, the 35M and 650M encoders reached 6.50 and 6.65°C MAE, respectively, both worse than the 8M FEP model. FEP labels remained beneficial within each size comparison, but the best absolute performance was obtained with the smallest encoder and FEP labels. Thus, in this low-Tm-data regime, the dominant gain came from the physical source label rather than from scaling the language model.

Encoder adaptation helps exploit the computed-label signal

We also asked whether the FEP gain required updating the sequence encoder. In a frozen-encoder control, overall errors increased, but FEP still improved over the frozen experimental-only baseline: the frozen experimental-only model achieved 7.38°C MAE, whereas frozen FEP achieved 7.20°C MAE. The improvement was smaller than in the fine-tuned-encoder setting, and the frozen FEP model remained worse than the fine-tuned-encoder FEP model. These results indicate that FEP labels carry useful information even when the representation is fixed, but limited encoder adaptation is important for extracting the strongest target-task benefit.

Physics-labeled generated variants connect transfer learning to design loops

Rosetta labels on variants proposed by the protein language model improved Tm prediction over the experimental-only baseline, whereas the comparison between language-model-proposed and random Rosetta-scored variants was suggestive but not conclusive. The paired improvement of language-model-proposed variants over experimental-only learning was 0.17°C (90% confidence interval, 0.03–0.31°C). By contrast, the paired difference between language-model-proposed and random variants was 0.07°C in favor of language-model-proposed variants, with a confidence interval crossing zero. We therefore interpret this result conservatively. Physics-labeled generated variants retain useful supervised signal for Tm prediction, but the present data do not prove that the current generator is superior to random variant sampling.

Mutation free-energy labels provide a physical interpretation

The final pattern suggests why FEP labels transfer to Tm prediction even though they are not Tm measurements. Experimental Tm labels provide sparse anchors for the absolute thermal-stability phenotype. Mutation free-energy labels provide local information about how stability changes under sequence perturbations. When learned jointly, these two views can shape a representation that is more useful for held-out Tm prediction than either sparse Tm labels alone or a generic simulation-derived summary. The negative result for the MD-derived Q-value defines the boundary of this interpretation: a simulation label must encode the right physical relation to the target phenotype.

Discussion

Source-label transfer is selective

This study asked how simulation-derived labels should be used when the experimental target is scarce and the simulated quantity is related to, but not identical with, that target. Using nanobody Tm prediction as a case study, we found that FEP mutation free-energy labels improved held-out experimental Tm prediction more clearly than any other source label tested. The result was not reproduced by a simple MD-derived Q-value label, nor was it explained by using a larger ESM2 encoder. These findings support a selective view of Sim2Real learning for biomolecules: simulation data are useful when the source label encodes physical information that transfers to the experimental phenotype.

Mutation free-energy labels connect relative and absolute stability

The main conceptual point is that the helpful source label did not measure Tm itself. FEP estimates mutation free-energy changes, whereas Tm is an experimental phenotype reflecting the temperature at which folded and unfolded states are balanced under a particular assay condition. These are not interchangeable quantities. Nevertheless, both describe the same underlying stability landscape from different perspectives. Sparse Tm measurements anchor absolute thermal-stability values for observed sequences, while mutation free-energy labels provide local directions in sequence space. The improvement from FEP therefore suggests that source labels can help even when they are not direct surrogates for the target, provided that they describe a physically relevant perturbation of the target property.

Scaling behavior depends on the source label

This result extends the materials-informatics Sim2Real argument to a more heterogeneous biomolecular setting. In materials applications, transfer learning often links computational and experimental estimates of related material properties, and scaling behavior can quantify the downstream value of a growing computational database (3). In the present protein case, the most useful computational labels were not additional Tm measurements but mutation free energies. The label-count sweep for FEP was consistent with a beneficial scaling trend but was not smooth enough to claim a universal biomolecular scaling law. The stronger conclusion is source dependence: simulation-derived data should be evaluated by target-task performance, and source labels must be chosen for transferable physical content rather than availability alone.

Model scale is not the main bottleneck

The encoder controls sharpen this conclusion. Prior work on PLMs has shown that fine-tuning can be critical for downstream protein-property prediction (16), and related nanobody Tm analyses suggest that larger pretrained ESM2 models do not automatically produce better Tm predictors. Our own size controls reached the same practical conclusion: 35M and 650M ESM2 encoders did not outperform the 8M encoder in either Tm-only or FEP-assisted settings. The best model used

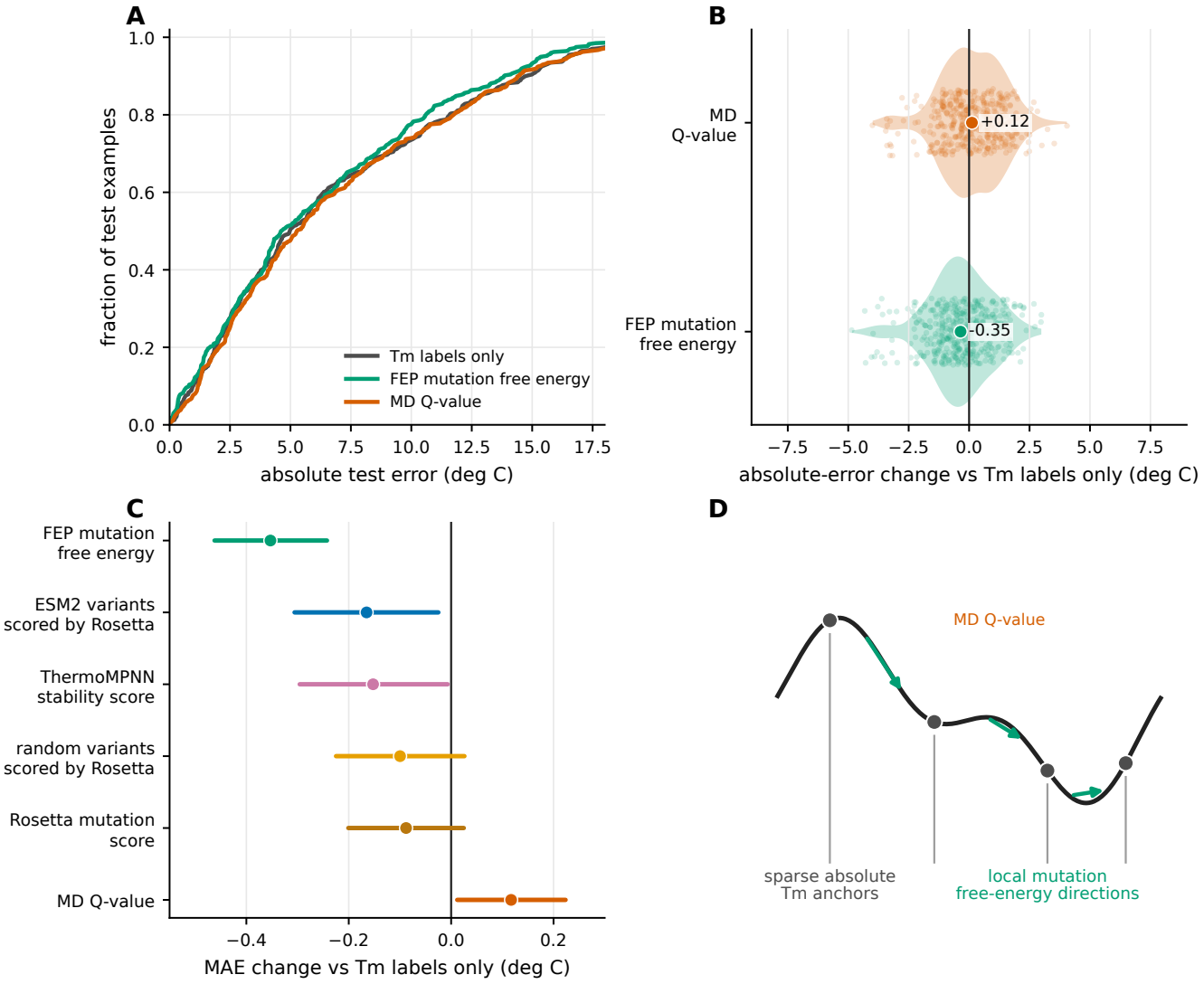

Fig. 4. Interpretation and boundary condition. (a) Cumulative distributions of absolute held-out Tm errors compare the experimental-only reference with FEP and MD Q-value source labels. (b) Per-example changes in absolute error relative to the experimental-only reference show where the FEP gain occurs. Negative values indicate lower error than the reference. (c) Paired ΔMAE estimates summarize all computed-label sources in the final comparison. Negative values indicate improvement over training on experimental Tm labels alone. (d) The interpretation is that sparse Tm measurements anchor absolute thermal stability, whereas mutation free-energy labels provide local stability directions; the MD Q-value control shows that simulation-derived labels must encode transferable physical content to improve the target task.

the 8M encoder with FEP source labels. Thus, for this dataset, the bottleneck was not simply language-model capacity. The more important ingredient was additional supervision carrying information about sequence-dependent stability changes.

Encoder adaptation helps but does not make every source useful

The frozen-encoder experiment further indicates how the source-label signal is used. FEP labels improved performance even when the encoder was fixed, showing that part of the signal can be captured by the existing sequence representation. However, the best FEP model required updating the encoder. This suggests that FEP labels help most when they are allowed to reshape the representation toward stability-relevant sequence directions while still being selected by experimental Tm performance. In practical terms, the result argues for controlled encoder adaptation rather than either fully fixed feature extraction or unconstrained source-task optimization.

The negative result for the MD-derived Q-value label is equally important. It shows that the failure mode is not a lack of simulation in general, but a mismatch between the source label and the target phenotype. The Q-value used here reports native-contact persistence in high-temperature trajectories, and may capture aspects of structural durability. However, it is a compressed trajectory statistic and may miss the mutation-specific energetic effects that determine how sequence changes alter the stability landscape. This distinction matters for future database design. A simulation campaign op-

timized for target transfer should not only maximize the number of labels; it should choose labels whose physical meaning is expected to transfer to the experimental decision being made.

Generated variants point toward design workflows

The generated-variant result points toward a possible design workflow. Rosetta scores assigned to ESM2-generated variants improved Tm prediction relative to Tm-only learning, although generated variants were not conclusively better than random variants in the current comparison. This should be viewed as a bridge, not a completed design loop. The positive signal indicates that proposed sequences can be labeled by physics-based calculations and used as supervised source data for a Tm predictor. A natural next step is an iterative loop in which a generator proposes variants, physics-based calculations annotate them, the Tm predictor is updated, and candidates are selected for experimental testing. The present work supplies evidence for the supervised learning component of that loop, but it does not test active learning, reinforcement learning, or experimental validation of newly proposed high-Tm nanobodies.

Limitations

Several limitations should guide interpretation. First, the experimental Tm training set is small, and the conclusions should be tested on additional nanobody collections and other protein families. Second, the source-label comparison is limited to the computational labels available here. Other free-energy protocols, stability predictors, trajectory summaries, or structure-based descriptors may transfer differently. Third, the FEP curve does not establish a clean power-law scaling relation; it supports a beneficial trend and a strong final improvement, but not a universal scaling exponent. Fourth, no new experimental measurements were performed to validate high-stability candidates. Finally, raw simulation trajectories may be too large to deposit fully, so reproducibility will depend on processed data, scripts, and clear reporting of the simulation protocol.

Outlook

For low-data experimental protein-property prediction, simulation labels should be treated as design choices rather than as generic data augmentation. The relevant question is not whether simulations can produce many labels, but whether those labels encode information that improves the target task under a target-centered validation protocol. In this study, mutation free-energy labels provided that information for nanobody Tm prediction. The broader strategy is therefore to use experimental data to define the target phenotype, use simulation to annotate physically meaningful perturbations around that phenotype, and evaluate the combined model only on held-out experimental data. This gives a disciplined path from simulation-assisted prediction toward future closed-loop protein engineering.

ACKNOWLEDGEMENTS

This work was supported by JSPS KAKENHI (Grant number: 23H03412) and JST NBDC (Grant number: JPMJND2603), and partly supported by MEXT as "Program for Promoting Researches on the Supercomputer Fugaku" (Development and application of large-scale simulation-based inferences for biomolecules JPMXP1020230119). Computational resources were provided by the supercomputer Fugaku at the RIKEN Center for Computational Science (Project IDs: hp230209, hp240215, hp250233).

AUTHOR CONTRIBUTIONS

Taihei Murakami and Kentaro Sasaki contributed equally to this work. Yasuhiro Matsunaga conceived and supervised the study, acquired funding, developed the analysis and training code, performed the analyses, interpreted the results, prepared the figures, and wrote the manuscript. Kentaro Sasaki performed computational calculations. Taihei Murakami contributed to draft writing. All authors reviewed and approved the final manuscript.

Declaration of competing interest

Taihei Murakami is an employee of Epsilon Molecular Engineering, Inc. Yasuhiro Matsunaga is a scientific advisor of Epsilon Molecular Engineering, Inc.

Data availability

Processed data required to reproduce the main and supplementary figures will be deposited in Zenodo at ZENODO_DATA_DOI_PLACEHOLDER. Raw simulation trajectories are large and will be made available at RAW_DATA_LOCATION_OR_REQUEST_POLICY_PLACEHOLDER.

Code availability

Analysis, training, and figure-generation code will be available on GitHub at GITHUB_REPOSITORY_URL_PLACEHOLDER. An archival release will be deposited in Zenodo at ZENODO_CODE_DOI_PLACEHOLDER.

AI-use disclosure

AI-assisted tools were used to support language editing, code development, and figure preparation. The authors reviewed and take responsibility for all manuscript content, analyses, and conclusions.

References

1. Hironao Yamada, Chang Liu, Stephen Wu, Yukinori Koyama, Shenghong Ju, Junichiro Shiomi, Junko Morikawa, and Ryo Yoshida. Predicting materials properties with little data using shotgun transfer learning. ACS Central Science, 5(10):1717–1730, 2019. doi: 10.1021/acscentsci.9b00804.
2. Yuta Aoki, Stephen Wu, Teruki Tsurimoto, Yoshihiro Hayashi, Shunya Minami, Tadamichi Okubo, Kazuya Shiratori, and Ryo Yoshida. Multitask machine learning to predict polymer–solvent miscibility using Flory–Huggins interaction parameters. Macromolecules, 56(14):5446–5456, 2023. doi: 10.1021/acs.macromol.2c02600.
3. Shunya Minami, Yoshihiro Hayashi, Stephen Wu, Kenji Fukumizu, Hiroki Sugisawa, Masashi Ishii, Isao Kuwajima, Kazuya Shiratori, and Ryo Yoshida. Scaling law of Sim2Real transfer learning in expanding computational materials databases for real-world predictions. npj Computational Materials, 11:146, 2025. doi: 10.1038/s41524-025-01606-5.
4. Serge Muyldermans. Nanobodies: Natural single-domain antibodies. Annual Review of Biochemistry, 82:775–797, 2013. doi: 10.1146/annurev-biochem-063011-092449.
5. Elena Alexander and Kam W. Leong. Discovery of nanobodies: a comprehensive review of their applications and potential over the past five years. Journal of Nanobiotechnology, 22(1):661, 2024. doi: 10.1186/s12951-024-02900-y.

6. Gert-Jan Bekker, Benson Ma, and Narutoshi Kamiya. Thermal stability of single-domain antibodies estimated by molecular dynamics simulations. Protein Science, 28(2):429–438, 2019. doi: 10.1002/pro.3546.
7. Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. Proceedings of the National Academy of Sciences of the United States of America, 118 (15):e2016239118, 2021. doi: 10.1073/pnas.2016239118.
8. Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. Science, 379 (6637):1123–1130, 2023. doi: 10.1126/science.ade2574.
9. Lasse M. Blaabjerg, Maher M. Kassem, Liam L. Good, Nicolas Jonsson, Matteo Cagiada, Kristoffer E. Johansson, Wouter Boomsma, Amelie Stein, and Kresten Lindorff-Larsen. Rapid protein stability prediction using deep learning representations. eLife, 12:e82593, 2023. doi: 10.7554/eLife.82593.
10. Mengyu Li, Hongzhao Wang, Zhenwu Yang, Longgui Zhang, and Yushan Zhu. DeepTM: A deep learning algorithm for prediction of melting temperature of thermophilic proteins directly from sequences. Computational and Structural Biotechnology Journal, 21:5544–5560, 2023. doi: 10.1016/j.csbj.2023.11.006.
11. J. A. E. Alvarez and S. N. Dean. TEMPRO: nanobody melting temperature estimation model using protein embeddings. Scientific Reports, 14:19074, 2024. doi: 10.1038/s41598-024-70101-6.
12. Kotaro Tsuboyama, Justas Dauparas, Jonathan Chen, Elodie Laine, Yasser Mohseni Behbahani, Jonathan J. Weinstein, Niall M. Mangan, Sergey Ovchinnikov, and Gabriel J. Rocklin. Mega-scale experimental analysis of protein folding stability in biology and design. Nature, 620 (7973):434–444, 2023. doi: 10.1038/s41586-023-06328-6.
13. Simon K. S. Chu, Kush Narang, and Justin B. Siegel. Protein stability prediction by fine-tuning a protein language model on a mega-scale dataset. PLOS Computational Biology, 20(7):e1012248, 2024. doi: 10.1371/journal.pcbi.1012248.
14. Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. Database, 2023:baad021, 2023. doi: 10.1093/database/baad021.
15. Yiming Zhang and Koji Tsuda. NbBench: Benchmarking language models for comprehensive nanobody tasks, 2025.
16. Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. Nature Communications, 15:7407, 2024. doi: 10.1038/s41467-024-51844-2.
17. James Dunbar, Konrad Krawczyk, Jinwoo Leem, Terry Baker, Angelika Fuchs, Guy Georges, Jiye Shi, and Charlotte M. Deane. SAbDab: the structural antibody database. Nucleic Acids Research, 42(D1):D1140–D1146, 2014. doi: 10.1093/nar/gkt1043.
18. Todd J. Dolinsky, Jens E. Nielsen, J. Andrew McCammon, and Nathan A. Baker. PDB2PQR: an automated pipeline for the setup of Poisson–Boltzmann electrostatics calculations. Nucleic Acids Research, 32(Web Server issue):W665–W667, 2004. doi: 10.1093/nar/gkh381.
19. Mats H. M. Olsson, Chresten R. Søndergaard, Michal Rostkowski, and Jan H. Jensen. PROPKA3: consistent treatment of internal and surface residues in empirical pK_a predictions. Journal of Chemical Theory and Computation, 7(2):525–537, 2011. doi: 10.1021/ct100578z.
20. David A. Case, Hasan Metin Aktulga, Kellon Belfon, David S. Cerutti, G. Andres Cisneros, Vinicius W. D. Cruzeiro, Negin Forouzesh, Timothy J. Giese, Andreas W. Götz, Holger Gohlke, Saeed Izadi, Koushik Kasavajhala, Mehmet C. Kaymak, Edward King, Tom Kurtzman, Tai-Sung Lee, Pengfei Li, Jian Liu, Tyler Luchko, Ray Luo, Madushanka Manathunga, Matias R. Machado, Hai Minh Nguyen, Kurt A. O'Hearn, Alexey V. Onufriev, Feng Pan, Sergio Pantano, Ruxi Qi, Ali Rahnamoun, Ali Risheh, Stephan Schott-Verdugo, Akhil Shajan, Jason M. Swails, Junmei Wang, Haixin Wei, Xiongwu Wu, Yongxian Wu, Shi Zhang, Shiji Zhao, Qiang Zhu, Thomas E. Cheatham, Daniel R. Roe, Adrian E. Roitberg, Carlos Simmerling, Darrin M. York, Maria C. Nagan, and Kenneth M. Merz. AmberTools. Journal of Chemical Information and Modeling, 63(20):6183–6191, 2023. doi: 10.1021/acs.jcim.3c01153.
21. Chuan Tian, Koushik Kasavajhala, Kellon A. A. Belfon, Lauren Raguette, He Huang, Angela N. Migues, John Bickel, Yuzhang Wang, Jorge Pincay, Qin Wu, and Carlos Simmerling. ff19SB: amino-acid-specific protein backbone parameters trained against quantum mechanics energy surfaces in solution. Journal of Chemical Theory and Computation, 16(1):528–552, 2020. doi: 10.1021/acs.jctc.9b00591.
22. Saeed Izadi, Ramu Anandakrishnan, and Alexey V. Onufriev. Building water models: a different approach. The Journal of Physical Chemistry Letters, 5(21):3863–3871, 2014. doi: 10.1021/jz501780a.
23. Peter Eastman, Jason Swails, John D. Chodera, Robert T. McGibbon, Yutong Zhao, Kyle A. Beauchamp, Lee-Ping Wang, Andrew C. Simmonett, Matthew P. Harrigan, Chaya D. Stern, Rafal P. Wiewiora, Bernard R. Brooks, and Vijay S. Pande. OpenMM 7: rapid development of high performance algorithms for molecular dynamics. PLOS Computational Biology, 13(7):e1005659, 2017. doi: 10.1371/journal.pcbi.1005659.
24. Robert T. McGibbon, Kyle A. Beauchamp, Matthew P. Harrigan, Christoph Klein, Jason M. Swails, Carlos X. Hernández, Christian R. Schwantes, Lee-Ping Wang, Thomas J. Lane, and Vijay S. Pande. MDTraj: a modern open library for the analysis of molecular dynamics trajectories. Biophysical Journal, 109(8):1528–1532, 2015. doi: 10.1016/j.bpj.2015.08.015.
25. Robert B. Best, Gerhard Hummer, and William A. Eaton. Native contacts determine protein folding mechanisms in atomistic simulations. Proceedings of the National Academy of Sciences of the United States of America, 110(44):17874–17879, 2013. doi: 10.1073/pnas.1311599110.
26. James C. Phillips, David J. Hardy, Julio D. C. Maia, John E. Stone, João V. Ribeiro, Rafael C. Bernardi, Ronak Buch, Giacomo Fiorin, Jérôme Hénin, Wei Jiang, Ryan McGreevy, Marcelo C. R. Melo, Brian K. Radak, Robert D. Skeel, Abhishek Singharoy, Yi Wang, Benoît Roux, Aleksei Aksimentiev, Zaida Luthey-Schulten, Laxmikant V. Kalé, Klaus Schulten, Christophe Chipot, and Emad Tajkhorshid. Scalable molecular dynamics on CPU and GPU architectures with NAMD. The Journal of Chemical Physics, 153(4):044130, 2020. doi: 10.1063/5.0014475.
27. William Humphrey, Andrew Dalke, and Klaus Schulten. VMD: Visual molecular dynamics. Journal of Molecular Graphics, 14(1):33–38, 1996. doi: 10.1016/0263-7855(96)00018-5.
28. Vijayaraj Ramadoss, François Dehez, and Christophe Chipot. AlaScan: A graphical user interface for alanine scanning free-energy calculations. Journal of Chemical Information and Modeling, 56(6):1122–1126, 2016. doi: 10.1021/acs.jcim.6b00162.
29. Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Ballard, Joshua Bambrick, Sebastian W. Bodenstein, David A. Evans, Chia-Chun Hung, Michael O'Neill, David Reiman, Kathryn Tunyasuvunakool, Zachary Wu, Akvilė Žemgulytė, Eirini Arvaniti, Charles Beattie, Ottavia Bertolli, Alex Bridgland, Alexey Cherepanov, Miles Congreve, Alexander I. Cowen-Rivers, Andrew Cowie, Michael Figurnov, Fabian B. Fuchs, Hannah Gladman, Rishub Jain, Yousuf A. Khan, Caroline M. R. Low, Kuba Perlin, Anna Potapenko, Pascal Savy, Sukhdeep Singh, Adrian Stecula, Ashok Thillaisundaram, Catherine Tong, Sergei Yakneen, Ellen D. Zhong, Michal Zielinski, Augustin Žídek, Victor Bapst, Pushmeet Kohli, Max Jaderberg, Demis Hassabis, and John M. Jumper. Accurate structure prediction of biomolecular interactions with AlphaFold 3. Nature, 630(8016):493–500, 2024. doi: 10.1038/s41586-024-07487-w.
30. Robert B. Best, Xiao Zhu, Jihyun Shim, Pedro E. M. Lopes, Jeetain Mittal, Michael Feig, and Alexander D. MacKerell. Optimization of the additive CHARMM all-atom protein force field targeting improved sampling of the backbone ϕ, ψ and side-chain χ_1 and χ_2 dihedral angles. Journal of Chemical Theory and Computation, 8(9):3257–3273, 2012. doi: 10.1021/ct300400x.
31. William L. Jorgensen, Jayaraman Chandrasekhar, Jeffry D. Madura, Roger W. Impey, and Michael L. Klein. Comparison of simple potential functions for simulating liquid water. The Journal of Chemical Physics, 79(2):926–935, 1983. doi: 10.1063/1.445869.
32. Jean-Paul Ryckaert, Giovanni Ciccotti, and Herman J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes. Journal of Computational Physics, 23(3):327–341, 1977. doi: 10.1016/0021-9991(77)90098-5.
33. Ulrich Essmann, Lalith Perera, Max L. Berkowitz, Tom Darden, Hsing Lee, and Lee G. Pedersen. A smooth particle mesh Ewald method. The Journal of Chemical Physics, 103(19):8577–8593, 1995. doi: 10.1063/1.470117.
34. Charles H. Bennett. Efficient estimation of free energy differences from Monte Carlo data. Journal of Computational Physics, 22(2):245–268, 1976. doi: 10.1016/0021-9991(76)90078-4.
35. Elizabeth H. Kellogg, Andrew Leaver-Fay, and David Baker. Role of conformational sampling in computing mutation-induced changes in protein structure and stability. Proteins: Structure, Function, and Bioinformatics, 79(3):830–838, 2011. doi: 10.1002/prot.22921.
36. Rebecca F. Alford, Andrew Leaver-Fay, Jeliazko R. Jeliazkov, Matthew J. O'Meara, Frank P. DiMaio, Hahnbeom Park, Maxim V. Shapovalov, P. Douglas Renfrew, Vikram K. Mulligan, Kalli Kappel, Jason W. Labonte, Michael S. Pacella, Richard Bonneau, Philip Bradley, Roland L. Dunbrack, Rhiju Das, David Baker, Brian Kuhlman, Tanja Kortemme, and Jeffrey J. Gray. The Rosetta all-atom energy function for macromolecular modeling and design. Journal of Chemical Theory and Computation, 13(6):3031–3048, 2017. doi: 10.1021/acs.jctc.7b00125.
37. Henry Dieckhaus, Michael Brocidiacono, Nicholas Z. Randolph, and Brian Kuhlman. Transfer learning to leverage larger datasets for improved prediction of protein stability changes. Proceedings of the National Academy of Sciences of the United States of America, 121(6):e2314853121, 2024. doi: 10.1073/pnas.2314853121.
38. Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In Proceedings of the IEEE Conference on Computer Vision and Pattern

Recognition, pages 7482–7491, 2018. doi: 10.1109/CVPR.2018.00781.

Supplementary Methods

Experimental target data and target-label scaling

All reported target-task comparisons use the same NbBench thermo-seq endpoint (15). The processed experimental files are data/nbbench/train.csv, data/nbbench/val.csv, and data/nbbench/test.csv. These files contain the columns text and label, where text is the amino-acid sequence and label is the melting temperature in degrees Celsius. The processed split sizes are 57 training sequences, 114 validation sequences, and 396 held-out test sequences.

The split definitions are fixed throughout the study. The 57 training examples are used to fit neural-network parameters. The validation split is used for checkpoint selection, early stopping, and selection among candidate model settings. The test split is held out until those choices have been fixed. The same rule is applied to Tm-only models and to models trained with computational source labels.

The training code transforms the experimental Tm values to a scaled regression target before optimization. In prepare.py, the scaler is fit only on the 57 training labels. The fitted transform is a RobustScaler followed by a MinMaxScaler with feature range [0,1]. Validation labels are transformed with this training-fitted scaler during model selection. Test predictions are averaged across seeds in scaled-label space and are then inverse-transformed to degrees Celsius before error metrics are computed. The validation and test label distributions therefore do not influence the fitted target-label transform.

The experimental-label scaling reference uses the same training split but subsamples it to the requested number of labels with the run seed. The validation and test splits are never subsampled. This experiment is used only as a positive control for the analysis pipeline: if more true target labels are added, the Tm prediction error should improve.

Processed computational source-label tables

Computational labels are loaded as source tasks rather than appended to the Tm data set as additional Tm measurements. Table 1 summarizes the processed target and source files used in the reported comparisons. Mutation-effect source tables use seq as the sequence column and ddg_scaled01 as the [0,1]-scaled source-label column. The MD-derived Q-value table also uses seq and ddg_scaled01 so that it can enter the same source-task loader; in that file, ddg_scaled01 is a min–max scaled Q-value, not a mutation-free-energy label.

Table 1. Processed target and source-label data. Row counts exclude header rows.

Data type	Role	Processed rows	Use in reported comparisons
Experimental Tm	Target train	57	Model fitting
Experimental Tm	Target validation	114	Checkpoint and setting selection
Experimental Tm	Target test	396	Final held-out evaluation
FEP mutation free energy	Source tasks	435 + 409	Main source task
Rosetta mutation score	Source tasks	435 + 409	Source comparison
ThermoMPNN stability score	Source tasks	435 + 409	Source comparison
Random variants scored by Rosetta	Source tasks	1000 + 1000	Source comparison
ESM2 variants scored by Rosetta	Source tasks	1000 + 1000	Source comparison
MD-derived Q-value	Source task	1143	Source comparison and scaling control

The source-task identifiers are assigned in the data loader. Experimental Tm is task 0. The two template-specific mutation-effect tables are tasks 1 and 2. The primary MD-derived Q-value source is task 3. The code also supports an optional second MD-derived source task, task 4, but the main figures use one MD-derived source at a time. The task identifier determines which scalar regression head is used and which task-specific loss term an example contributes to.

Exact sequence overlap was checked between the processed source-label tables and the fixed NbBench splits. The processed FEP/Rosetta mutation-effect tables, random variants scored by Rosetta, and ESM2 variants scored by Rosetta had no exact sequence matches to the NbBench training, validation, or test sequences. The ThermoMPNN tables used the same template-specific variant design as the mutation-effect sources. The SAbDab-derived MD Q-value table contained exact matches to 2 NbBench training sequences, 4 validation sequences, and 8 test sequences. This overlap is disclosed because it affects interpretation of the MD control; it cannot explain the positive FEP result, and the MD Q-value source did not improve held-out Tm prediction.

For mutation-effect experiments with requested source-label count n, the loader samples up to n rows independently from each of the two template tables with random_state=seed. Thus, the final FEP setting with $n = 320$ supplies 320 sampled 1MEL rows and 320 sampled 4IDL rows before the source-task train/validation split. For MD-derived Q-value experiments, the loader samples n rows from the single processed Q-value table. Source rows are split

80/20 into source-training and source-validation rows using the same seed. The production training call then filters the validation data passed to the trainer so that checkpoint selection and early stopping are based only on the experimental Tm validation examples.

SAbDab-derived nanobody structure panel for MD

The MD source-label panel was constructed from SAbDab, the Structural Antibody Database (17). The local SAbDab nanobody summary file /home/yasu/tmp/mdclaw/sabdab_nano_summary_all.tsv contains 2422 rows with PDB identifier, heavy-chain assignment, light-chain assignment, model index, antigen annotation, experimental method, resolution, species, and other SAbDab fields. The manifest construction script /home/yasu/tmp/mdclaw/scripts/build_manifest_nano_imgt.py matches this summary against IMGT-renumbered PDB files in /home/yasu/tmp/mdclaw/nanobodies/imgt/.

The selection rule is deterministic. For each PDB identifier with both a SAbDab summary row and a local IMGT-renumbered PDB file, the script picks the first heavy-chain row encountered in the SAbDab table. The chain is accepted only if the same chain identifier is present in the IMGT PDB file. The output manifest contains one entry per PDB identifier, with fields for the manifest row number, PDB identifier, selected heavy chain, SAbDab TSV row, IMGT PDB path, and MDClaw job directory. This initial one-per-PDB manifest contained 1177 structures.

The 400 K MD campaign reused the systems prepared in the 300 K campaign and created a 1146-entry 400 K manifest. The selected 400 K manifest consisted of 798 X-ray structures, 345 electron-microscopy structures, and 3 NMR structures according to the SAbDab method field. All selected rows had Lchain=NA in the SAbDab summary, consistent with a single-domain antibody panel. After trajectory and Q-value processing, 1143 unique PDB identifiers yielded usable source-label rows.

MDClaw system preparation and simulation protocol

Each selected structure was prepared as a nanobody-only MD system. The script /home/yasu/tmp/mdclaw/scripts/run_prep_one_imgt.sh registers the IMGT-renumbered PDB as a local structure source, creates an MDClaw fetch node, extracts the selected heavy chain, includes only protein atoms, and leaves termini uncapped. The preparation call uses pH 7.4. MDClaw cleans protein chains with PDBFixer and applies pH-aware protonation through pdb2pqr/PROPKA when available (18, 19); its fallback path uses AmberTools-based hydrogen assignment and Amber naming. The resulting prepared chain is solvated with explicit OPC water, 15 Å padding, and 0.15 M NaCl. Amber topology and coordinate files are built with ff19SB and OPC (20–22).

The base 300 K simulation branch uses OpenMM on CUDA devices (23). It runs 1 ns of NVT equilibration followed by 1 ns of NPT equilibration at 300 K and 1 atm, with a 2-fs timestep and no hydrogen-mass repartitioning. Production at 300 K was generated for the broader MD campaign, but the source label used in this manuscript comes from the 400 K branch.

The 400 K branch is encoded in the MDClaw batch scripts nano_eq_002_400K.sbatch, nano_eq_003_400K.sbatch, and nano_prod_002_400K.sbatch. The first 400 K relaxation node restarts from the 300 K equilibrated state and runs 1 ns of NVT at 400 K with Cα restraints. The second relaxation node restarts from that state and repeats 1 ns of 400 K NVT with Cα restraints. Both relaxation stages use a 2-fs timestep and no hydrogen-mass repartitioning. The production node runs 100 ns of restraint-free 400 K NVT, with hydrogen-mass repartitioning, 4 amu hydrogen mass, a 4-fs timestep, and coordinate/energy output every 100 ps.

In the MDClaw OpenMM implementation, explicit-solvent periodic systems use PME electrostatics with a 1.0-nm nonbonded cutoff and constraints on bonds involving hydrogen atoms. NPT stages use a Monte Carlo barostat. NVT stages omit the barostat. The integrator is the Langevin middle integrator with friction 1 ps^{-1}. The 100-ps reporting interval gives 1000 expected frames for a complete 100 ns production trajectory.

MD-derived Q-value extraction

The processed MD source table was generated with scripts/extract_q_values.py from the sim2real repository. The script scans /home/yasu/tmp/mdclaw/job_nano_* job directories and uses the 400 K production node prod_002. A job is eligible only if it contains a production trajectory nodes/prod_002/artifacts/trajectory.dcd, a prepared reference PDB nodes/prep_001/artifacts/merge/merged.pdb, and an Amber parameter file nodes/topo_001/artifacts/system.parm7. The amino-acid sequence is extracted from the prepared PDB by reading the first Cα atom for each residue. Sequences shorter than 50 residues are excluded.

Q-values are computed with MDTraj (24). The topology is loaded from the Amber parameter file, and the native reference is frame 0 of the production trajectory. The contact calculation uses the atom selection protein and backbone and not element H. Candidate native contacts must be separated by more than three residues, have a reference-frame distance below 0.45 nm, and include at least one atom from a hydrophilic residue (Asp, Glu, Gln,

Asn, Arg, Lys, or His, including Amber protonation variants). For a native-contact pair (i, j) with reference distance r_{ij}^0 and instantaneous distance r_{ij}, the per-frame contribution is

$$q_{ij} = \frac{1}{1 + \exp\{\beta(r_{ij} - \lambda r_{ij}^0)\}},$$

with $\beta = 50\ \mathrm{nm}^{-1}$ and $\lambda = 1.8$, following the smooth native-contact form used in Best–Hummer-style Q-value analyses (25). The frame-level Q-value is the mean of q_{ij} over all selected native contacts.

The trajectory-level label is the mean Q-value over the terminal portion of the production trajectory. For trajectories with more than 300 frames, the implementation uses the larger of 300 frames and one third of the available frames; for shorter trajectories it uses all available frames. For complete 100 ns trajectories with 100-ps output, this corresponds to 333 terminal frames, approximately the final 33 ns. In the processed 400 K Q-value table, the number of frames used ranges from 66 to 333 with median 333. The number of native contacts ranges from 28 to 728 with median 173. The sequence length range is 58–461 residues with median 122. Raw Q-values range from 0.0437 to 0.9994. These raw Q-values are min–max scaled to [0,1] and stored in the loader-facing column ddg_scaled01.

Sequence tokenization and encoder inputs

Sequences are tokenized with the ESM2 tokenizer associated with the selected encoder (8). The code uses truncation and fixed-length padding to MAX_LENGTH = 160 residues. The tokenized fields are input_ids and attention_mask. The source-task column task is renamed to task_ids before training. For the production protocol, the trainer receives all target and sampled source rows for training, but its evaluation data are filtered to rows with task_ids == 0, which are the experimental Tm validation rows.

The fine-tuned-encoder setting keeps token IDs and attention masks in the training data and updates the ESM2 weights. The frozen-encoder setting calls prepare.precompute_embeddings before training. In that setting, ESM2 is run once for each target and source sequence, the first-token embedding is stored as an embedding field, and the subsequent optimizer updates only the regression trunk and task heads.

Neural-network architecture

The default encoder is facebook/esm2_t6_8M_UR50D. The model takes the first-token ESM2 representation as the sequence-level embedding. The shared regression trunk has dimensions hidden-size $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 32, with ReLU activations and dropout after the first two hidden layers. The Tm head maps the 32-dimensional representation to one scaled scalar prediction.

The Tm-only baseline consists of the ESM2 encoder, the shared trunk, and the Tm head. Source-transfer models keep this target path and add source heads. The main mutation-effect model uses separate source heads for the two template-specific tasks. The code also implements source-head controls with a single shared mutation-effect head, a template-conditioned head, and a template-specific affine calibration on top of a shared head. These controls test whether the two template-specific mutation-effect sources require separate output calibration.

Architecture controls in train.py include residual, latent, and mixture-style source-fusion variants. In these variants, the source path can produce a correction to the Tm prediction, a latent interaction between Tm and source features, or a gate between a target-only expert and a source-informed expert. For the main reported FEP result, the selected model is the original shared-trunk architecture with separate template-specific source heads. The architecture controls are retained as sensitivity analyses rather than as the main claim.

Training objective, optimizer, and checkpoint selection

Training examples are represented by sequence, scaled label, and task identifier. Each example contributes loss only through the regression head corresponding to its task. The per-task loss is Huber loss with $\delta = 1.0$ on the scaled label. The default multi-task objective uses learned task-uncertainty weights (38). For task k with loss L_k and learned log-scale s_k, the loss contribution is

$$\frac{L_k}{2\exp(2s_k)} + s_k.$$

The total loss is the sum of the contributions from the tasks present in the minibatch. Fixed source-loss weights are also implemented and were included in the candidate-setting searches for source classes where loss balance could change validation performance.

The optimizer is AdamW. The default head/trunk learning rate is 3×10^{-4}, the default fine-tuned-encoder learning rate is 1×10^{-4}, weight decay is 0.04, dropout is 0.15, batch size is 16, warmup is 100 steps, and the schedule is cosine decay. Training runs for at most 400 epochs. Checkpoints are evaluated and saved once per epoch. Mixed precision is enabled when CUDA is available. In the fine-tuned-encoder setting, the custom trainer creates separate optimizer

parameter groups for ESM2 parameters and newly initialized trunk/head parameters. In the frozen-encoder setting, ESM2 parameters are non-trainable and no encoder optimizer group is used.
Within each run, the best checkpoint is the checkpoint with the lowest mean squared error on the experimental Tm validation rows. Early stopping monitors the same validation metric with patience 15 and threshold 0. Source-task validation rows are not passed to the trainer in the production protocol. This implementation detail is central for the comparison: source labels influence the learned representation through training loss, but they do not decide which checkpoint is used for target-task prediction.

Candidate-setting selection and final evaluation

Candidate settings are selected on the experimental Tm validation split. The common candidate grid includes the default setting; encoder learning rates of 3×10^{-5}, 1×10^{-4}, and 3×10^{-4}; a lower head learning rate of 1×10^{-4} paired with encoder learning rate 3×10^{-5}; and dropout rates 0.05, 0.15, and 0.30. Fixed source-loss weights are added for source classes where the source label scale may require explicit balancing. Tm-only candidates include shared, residual, and latent architecture variants. Mutation-effect source candidates use the shared-trunk family for the main source comparison, with a separate source-head design comparison. MD-derived source candidates include fixed-weight settings because Q-value labels differ in scale and interpretation from mutation-free-energy labels.
Each candidate is first evaluated as a three-seed ensemble on the Tm validation split. The selected setting for each source class is then evaluated on the held-out test split as a ten-seed ensemble. Final comparisons use seeds 1–10. The seed controls Python random, NumPy, PyTorch, and CUDA randomness when CUDA is available; it also controls target-training subsampling in the target-label scaling experiment, source-row sampling, and source train/validation splitting. The label-count curves use fixed source-specific training recipes while the number of source labels is varied.

Statistics and stored outputs

The primary metric is MAE in degrees Celsius on the 396 held-out test examples. For each condition, the ten seeded models produce scaled Tm predictions for every test sequence. Predictions are averaged in scaled-label space and then inverse-transformed to degrees Celsius. Root mean squared error, coefficient of determination, Spearman correlation, and Pearson correlation are also computed by the evaluation code, but the figures emphasize MAE because it is directly interpretable as average temperature error.
Single-condition confidence intervals are estimated by nonparametric bootstrap resampling over test examples. The absolute-error vector from the ten-seed ensemble is resampled with replacement 10,000 times using a fixed bootstrap seed. The reported 90% interval is the 5th to 95th percentile range of the bootstrapped MAE values. Paired comparisons use the same bootstrap indices for the candidate and reference absolute-error vectors. The reported ΔMAE is candidate minus reference, so negative values indicate lower error than the reference.
Each call to prepare.py writes a structured scaling.json file in the result directory. This file records command-line arguments, relevant environment variables, resolved hyperparameters, per-label-count metrics, bootstrap intervals, per-example absolute errors, and paired bootstrap summaries. The main source comparison is summarized by results/source_screen/final_source_screen_summary.json. The frozen-encoder source comparison is summarized by results/source_screen/final_frozen_core_summary.json. The source-head controls are summarized by results/ddg_head_search/final_ddg_head_summary.json. Additional source-combination controls are summarized by results/abcd_search/final_abcd_summary.json. The figure set is assembled by plot/make_outline_figures.py, which also writes paper/tex/figures/outline_figure_source_screen.tsv.

Interpreting the label-count curves

The target-label scaling curve is an internal check on the pipeline. It asks whether the same model family and evaluation code detect the expected gain when more true experimental Tm labels are available. The FEP source-label curve asks a different question: whether more labels from a physically related but different computed quantity can improve the same held-out Tm endpoint. The MD Q-value curve asks whether adding more simulation-derived labels is sufficient when the source label measures a more indirect structural response. The important comparison is therefore not simply "simulation versus no simulation." It is whether the source label carries the kind of physical information that transfers to the target property under target-centered model selection.

Supplementary figure source tables

The supplementary figures are generated from tabular source files stored in paper/analysis/supplementary/tables/. These tables are written by plot/make_supplementary_figures.py from the same result summaries used for the main figures. The corresponding rendered figures are stored in paper/analysis/supplementary/figures/ and copied to paper/tex/figures/ for typesetting.

Supplementary Figures

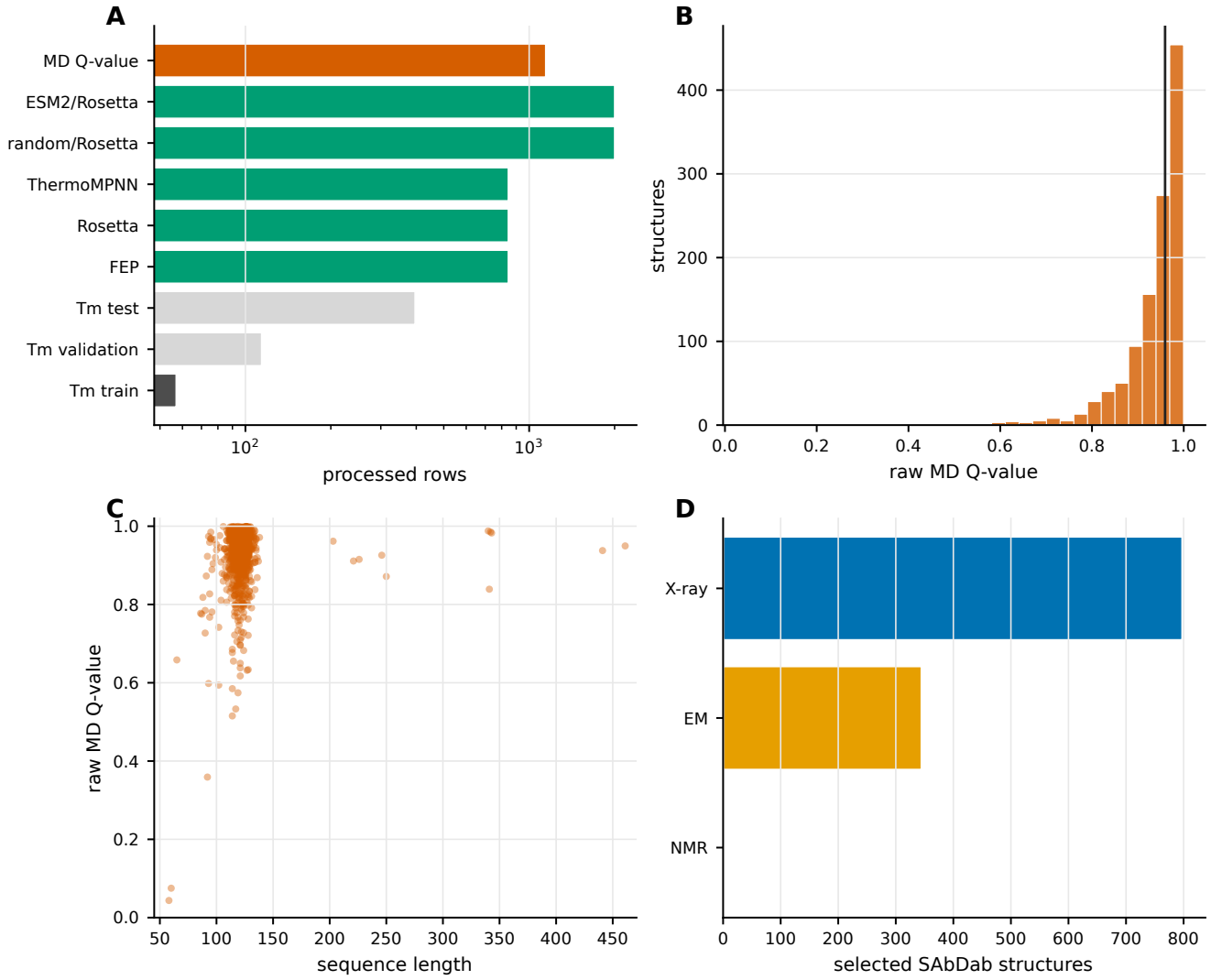

Fig. 5. Supplementary Figure 1. Processed data sets and MD-derived Q-value labels. (a) Processed target and source-label row counts used in the reported experiments. (b) Distribution of raw MD-derived Q-values before min–max scaling. (c) Relationship between sequence length and raw MD-derived Q-value. (d) Experimental methods for the selected SAbDab structures used in the 400 K MD panel.

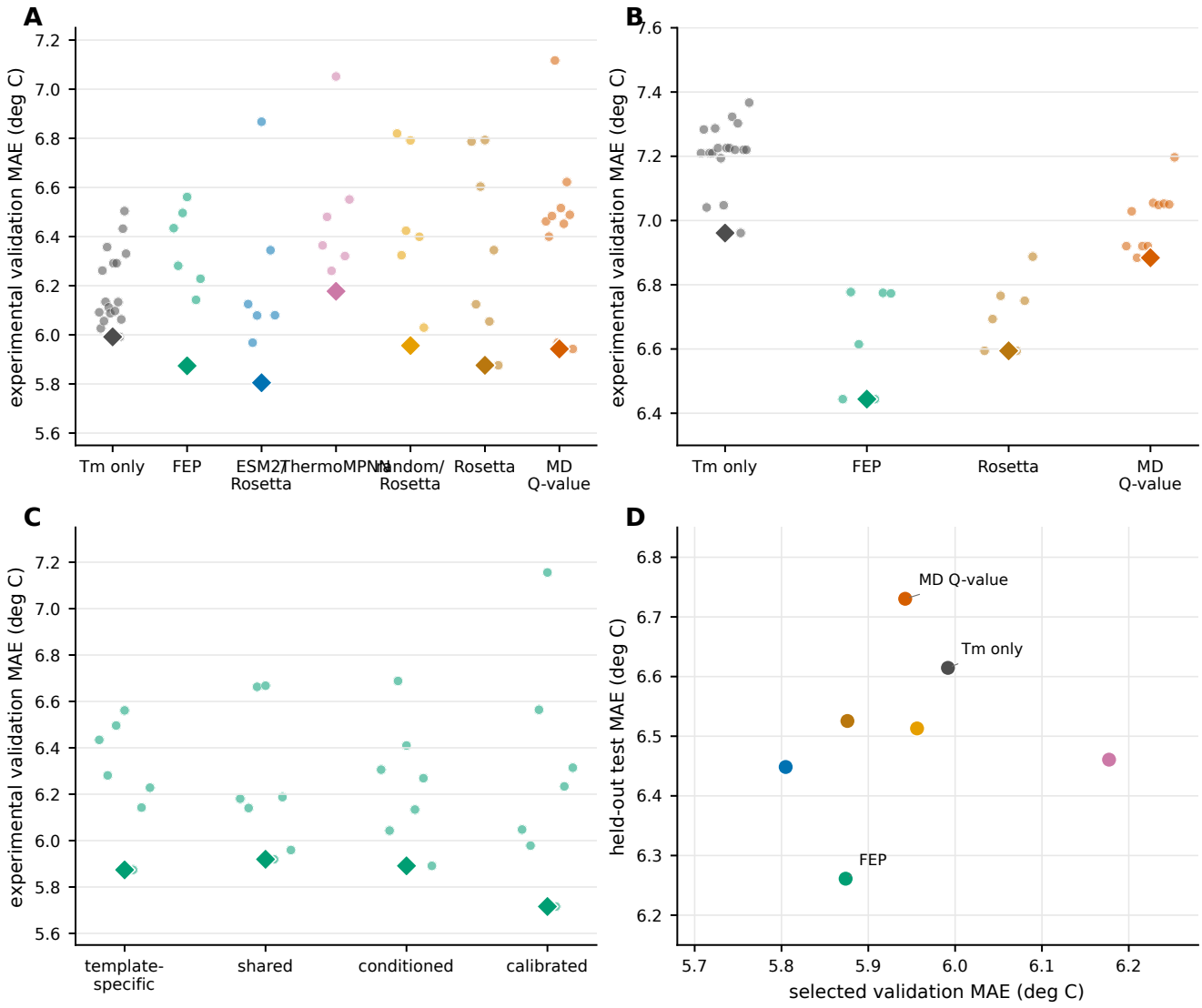

Fig. 6. Supplementary Figure 2. Candidate-setting searches and target-validation selection. (a) Experimental validation MAE values for fine-tuned-encoder source comparisons. (b) Experimental validation MAE values for frozen-encoder controls. (c) Candidate source-head designs for FEP mutation free-energy labels. (d) Relationship between selected validation MAE and held-out test MAE for the final source comparison. Diamonds mark the best validation setting in panels a–c.

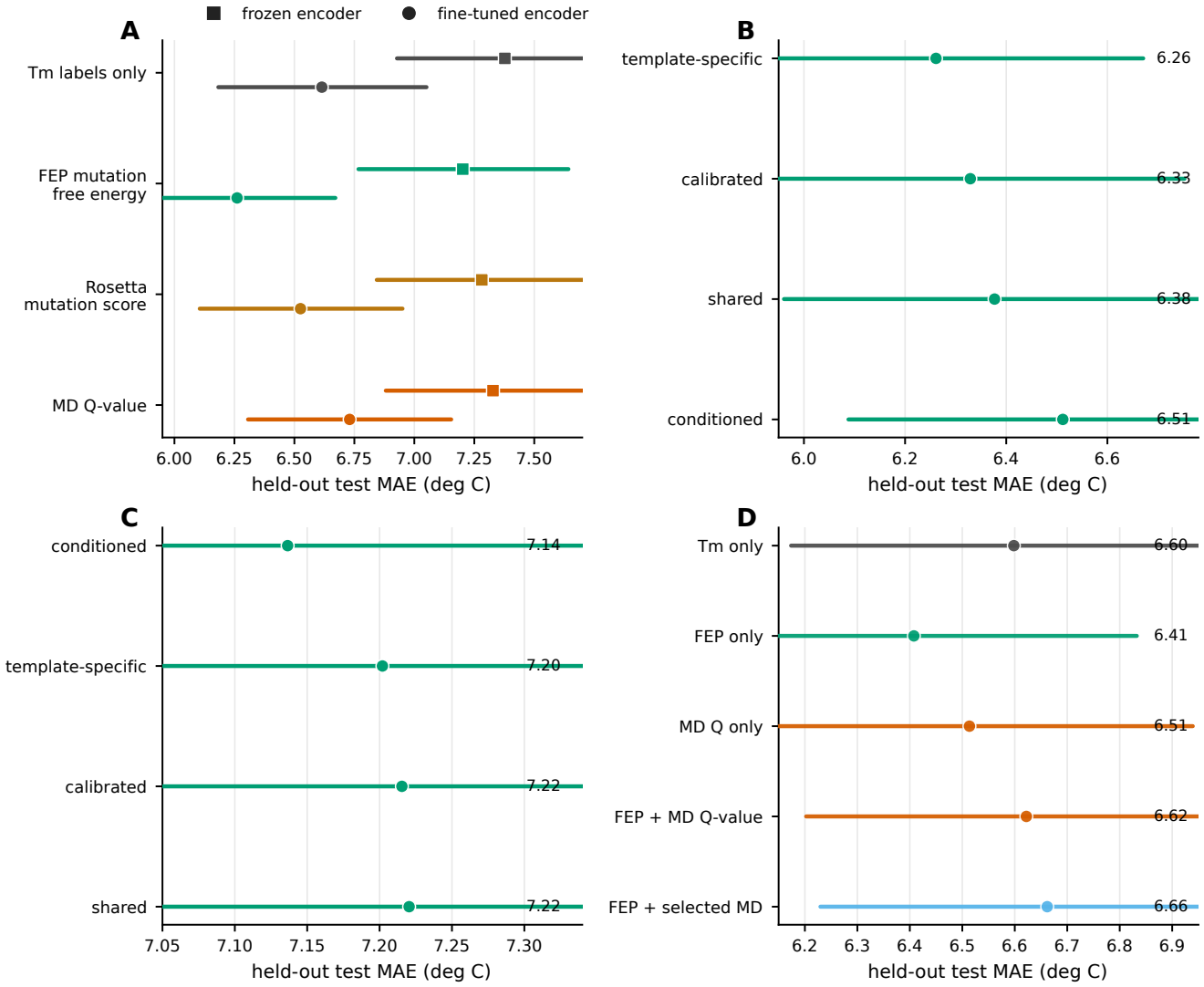

Fig. 7. Supplementary Figure 3. Model and source-combination controls. (a) Frozen-encoder and fine-tuned-encoder comparisons for selected source labels. (b) Fine-tuned-encoder FEP source-head controls. (c) Frozen-encoder FEP source-head controls. (d) Source-combination controls comparing Tm-only, FEP-only, MD-Q-only, and combined-source settings.

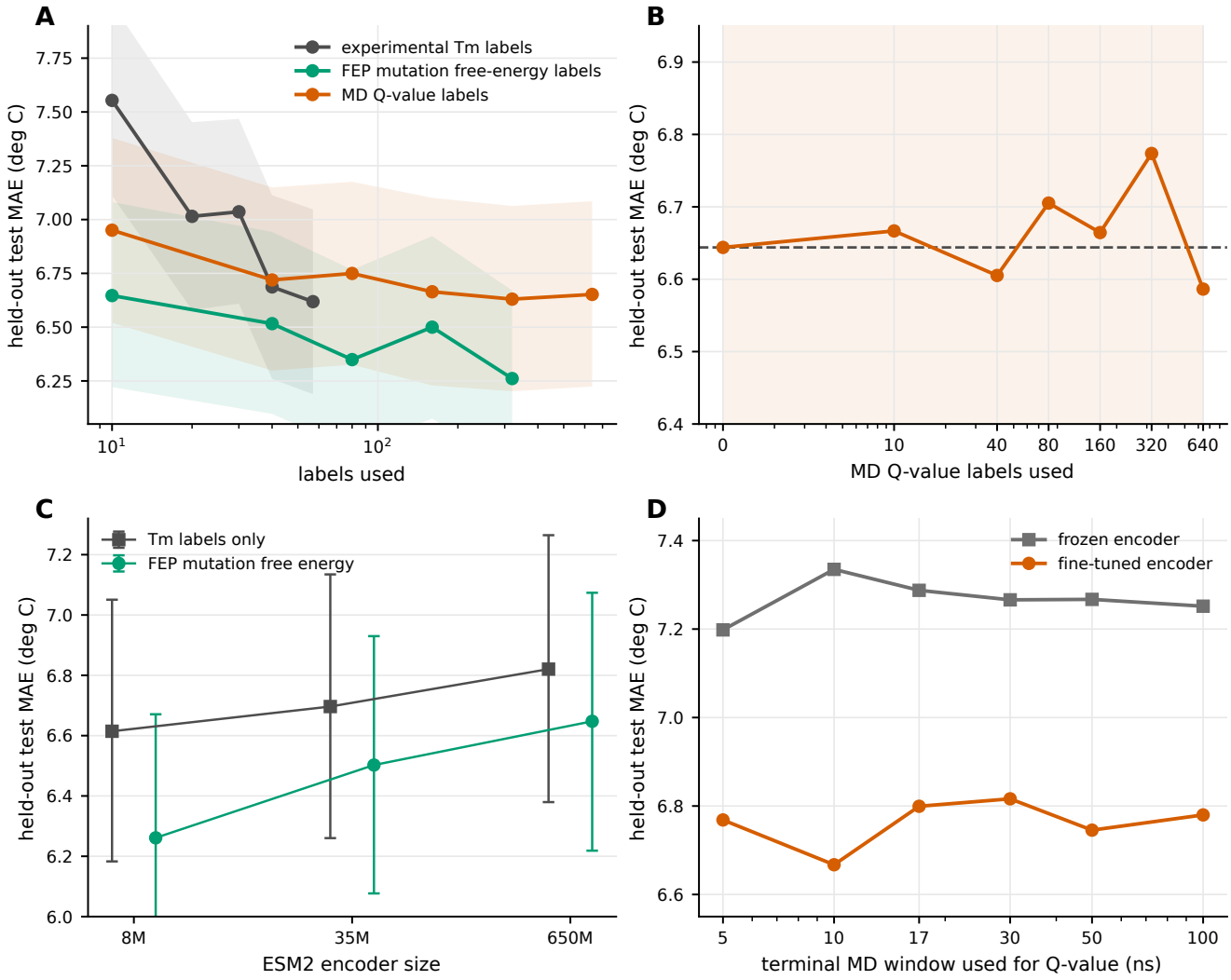

Fig. 8. Supplementary Figure 4. Scaling, encoder-size, and MD-window controls. (a) Label-count curves for experimental Tm labels, FEP labels, and MD Q-value labels. (b) MD Q-value label-count curve after selecting the candidate setting separately for each label count on the experimental validation split. (c) ESM2 encoder-size controls for Tm-only and FEP settings. (d) MD trajectory-window controls for Q-value labels.

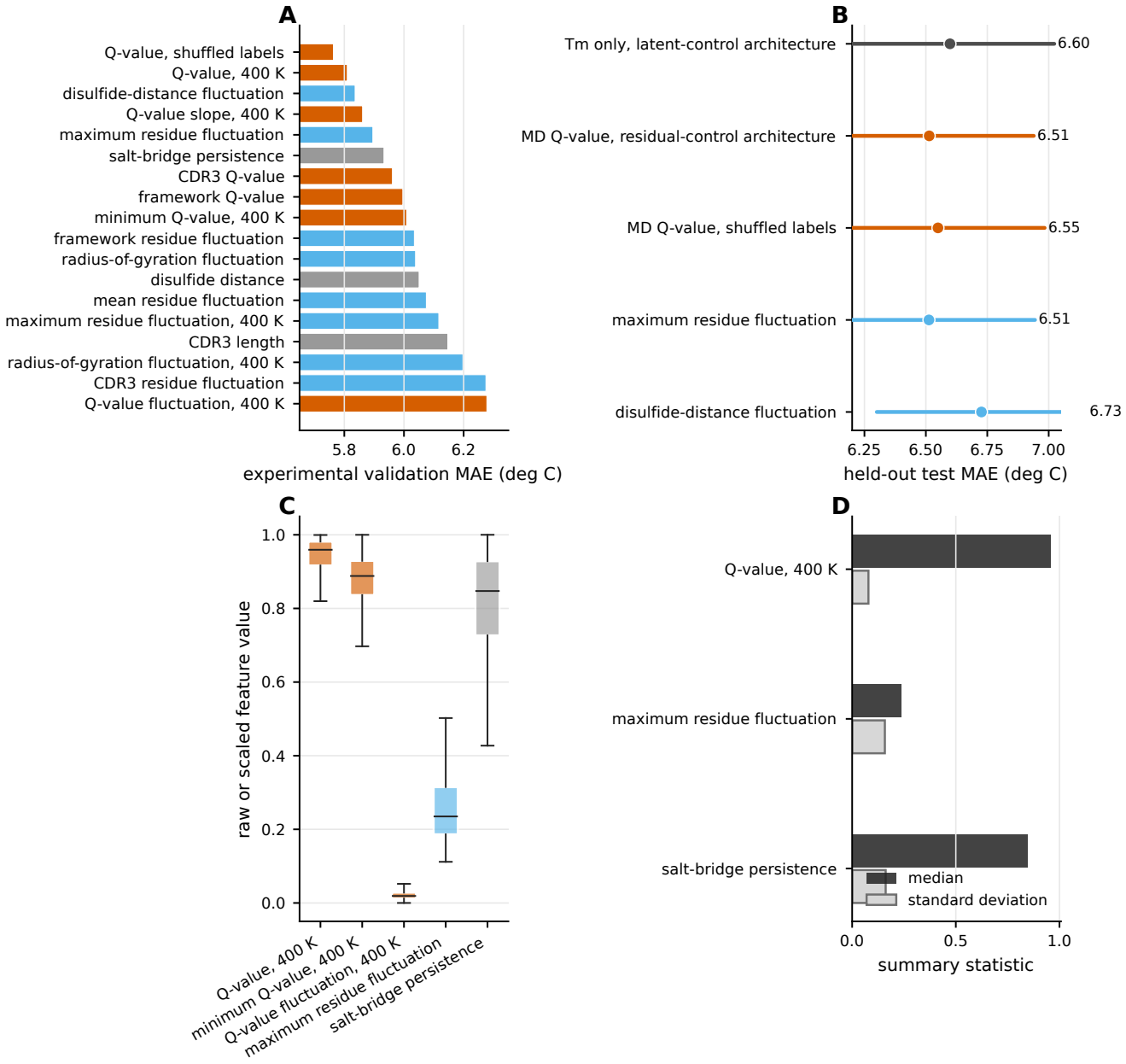

Fig. 9. Supplementary Figure 5. MD feature and architecture controls. (a) Experimental validation MAE values for alternative MD-derived features. (b) Held-out test MAE values for selected architecture and shuffled label controls. (c) Distributions of representative MD-derived feature values. (d) Summary statistics for representative MD-derived feature values.